DATA_CLEANING

✅ Healthcare Data Cleaning & Analysis (Python Project)

📌 Import Libraries & Load Data

```
import pandas as pd
import numpy as np

df = pd.read_csv("healthcare_data_cleaning_dataset.csv")

# Basic info
print(df.info())
print(df.head())
```

🔹 Q1. Missing Data Identification

```
# Missing values count
missing_counts = df.isnull().sum()

# Missing percentage
missing_percentage = (missing_counts / len(df)) * 100

print("Missing Values:\n", missing_counts)
print("\nMissing Percentage:\n", missing_percentage)
```

✅ **Output Insight:**

- Age → ~11.76% missing
- Treatment_Cost → ~11.63% missing

🔹 Q2. Handling Missing Age

```
# Use MEDIAN (robust to outliers)
median_age = df['Age'].median()

df['Age'].fillna(median_age, inplace=True)
```

✅ **Justification:**

- Median is better than mean because **age may contain extreme values**
- Prevents distortion

🔹 **Q3. Handling Missing Treatment Cost**

```
# Highly skewed → use MEDIAN
median_cost = df['Treatment_Cost'].median()

df['Treatment_Cost'].fillna(median_cost, inplace=True)
```

✅ **Why Median?**

- Cost is **right-skewed (expensive treatments)**
- Mean would inflate values

🔹 **Q4. Duplicate Patient Records**

```
# Before removal
before = df.shape[0]

# Remove duplicates
df = df.drop_duplicates()

# After removal
after = df.shape[0]

print("Before:", before)
print("After:", after)
print("Duplicates Removed:", before - after)
```

🔹 **Q5. Invalid Age Values**

```
# Detect invalid ages
invalid_age = df[(df['Age'] < 0) | (df['Age'] > 100)]
```

```
print("Invalid Age Records:\n", invalid_age)

# Remove them
df = df[(df['Age'] >= 0) & (df['Age'] <= 100)]
```

✅ **Decision:**

- Removed because unrealistic data harms analysis

🔹 **Q6. Outlier Detection (IQR Method)**

```
Q1 = df['Treatment_Cost'].quantile(0.25)
Q3 = df['Treatment_Cost'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df['Treatment_Cost'] < lower_bound) |
              (df['Treatment_Cost'] > upper_bound)]

print("Number of Outliers:", len(outliers))
```

🔹 **Q7. Outlier Treatment (Winsorization)**

```
# 5th and 95th percentile
lower_cap = df['Treatment_Cost'].quantile(0.05)
upper_cap = df['Treatment_Cost'].quantile(0.95)

df['Treatment_Cost'] = np.where(df['Treatment_Cost'] < lower_cap, lower_cap,
                       np.where(df['Treatment_Cost'] > upper_cap, upper_cap,
                                df['Treatment_Cost']))
```

✅ **Why Winsorization?**

- Keeps all records (business requirement)

- Limits extreme impact

🔹 Q8. Transformation (Log Transformation)

```
# Create new column
df['Log_Treatment_Cost'] = np.log(df['Treatment_Cost'])

# Compare distributions
print(df[['Treatment_Cost', 'Log_Treatment_Cost']].describe())
```

✅ **Benefit:**

- Reduces skewness
- Makes data more normal for analysis

🔹 Q9. Time-Based Missing Handling

```
# Convert to datetime
df['Admission_Date'] = pd.to_datetime(df['Admission_Date'])

# Sort by date
df = df.sort_values(by='Admission_Date')

# Forward fill
df.fillna(method='ffill', inplace=True)

# Backward fill (optional)
df.fillna(method='bfill', inplace=True)
```

✅ **Justification:**

- Dates follow sequence → nearby values are meaningful
- Forward fill maintains timeline consistency
